

Barrier

Самый распространенный способ синхронизации в OpenMP – барьер. Он оформляется с помощью директивы `barrier`.

```
#pragma omp barrier
```

Нити, выполняющие текущую параллельную область, дойдя до этой директивы, останавливаются и ждут, пока все нити не дойдут до этой точки программы, после чего разблокируются и продолжают работать дальше. Кроме того, для разблокировки необходимо, чтобы все синхронизируемые нити завершили все порождённые ими задачи (`task`).

```
#include <stdio.h>
#include <omp.h>

int main()
{
    #pragma omp parallel
    {
        printf( "До барьера: Поток %d, Сообщение 1\n" ,
omp_get_thread_num( ) );
        printf( "До барьера: Поток %d, Сообщение 2\n" ,
omp_get_thread_num( ) );

        #pragma omp barrier

        printf( "После барьера: Поток %d, Сообщение 3\n" ,
omp_get_thread_num( ) );
    }
}
```

```
int main()
{
    int a[5], i;

    #pragma omp parallel
    {
        #pragma omp for
        for (i = 0; i < 5; i++){
            a[i] = i * i;
            printf( "До барьера: Поток %d, a[%d] = %d \n", omp_get_thread_num(), i, a[i]);
        }

        #pragma omp master
        for (i = 0; i < 5; i++){
            printf( "До барьера: Поток %d , a[%d] = %d \n", omp_get_thread_num(), i, a[i]);
        }

        #pragma omp barrier

        #pragma omp for
        for (i = 0; i < 5; i++){
            a[i] += i;
            printf( "После барьера: Поток %d, идентификатор %d\n", omp_get_thread_num(), i);
        }
    }
}
```

Ordered

Директива `ordered` определяет блок внутри тела цикла, который должен выполняться в том порядке, в котором итерации идут в последовательном цикле.

```
#pragma omp ordered
```

Относится к самому внутреннему из объемлющих циклов, а в параллельном цикле должна быть задана опция `ordered`. Нить, выполняющая первую итерацию цикла, выполняет операции данного блока. Нить, выполняющая любую следующую итерацию, должна сначала дождаться выполнения всех операций блока всеми нитями, выполняющими предыдущие итерации. Может использоваться, например, для упорядочения вывода от параллельных нитей.

Исполнение внутреннего блока `ordered` будет подчиняться естественному порядку цикла, в то время как код извне будет исполняться параллельно.

```
#pragma omp parallel for ordered
for ( int i = 0 ; i < 5 ; i++) {
    printf( "Нить %d, итерация %d\n",
omp_get_thread_num( ), i );
    #pragma omp ordered
    {
        printf( "ordered: Нить %d, итерация
%d\n", omp_get_thread_num( ), i );
    }
}
```

Critical

С помощью директивы `critical` создаётся критическая секция, вход в которую синхронизировано для нескольких потоков.

```
#pragma omp critical [ (<имя_критической_секции> ) ]
```

В каждый момент времени в критической секции может находиться не более одной нити. Все другие нити, выполнившие директиву для секции с данным именем, будут заблокированы, пока вошедшая нить не закончит выполнение. Как только работавшая нить выйдет из критической секции, одна из заблокированных нитей войдет в неё. Если на входе критической секции ожидают несколько нитей, то случайным образом выбирается одна из них, остальные продолжают ждать.

Критические секции могут быть именованными или не именованными. Согласно стандарту все критические секции без имени, будут ассоциированы одним именем. Присвоение имени позволит одновременно выполнять две и более критические секции.

```
#pragma omp critical
{
    doworkA( );
}
#pragma omp critical
{
    doworkB( );
}
// будет завершена секция с doworkA() , а только потом с doworkB()
```

...

```
#pragma omp critical (first)
{
    doworkA( );
}
#pragma omp critical (second)
{
    doworkB( );
}
```

//секции с doworkA () и doworkB() могут быть выполнены одновременно в двух потоках

Критические секции удобный инструмент для разграничения доступа к общим данным и избежания состояний гонки. Однако, критическая секция привносит в параллельную программу последовательный код, что снижает эффективность.

Flush

Для OpenMP характерно, что каждая нить имеет собственное временное представление памяти, что допускает временную несогласованность между представлением памяти нити и общей памятью. Такая модель памяти таит в себе много неочевидных ловушек. Если в последовательной программе нить сначала записывает новое значение переменной, а затем использует его по чтению, то аппаратура и компилятор гарантируют, что использовать мы будем новое значение. Если же мы имеем дело с конкурентным выполнением записи в общую переменную одной нитью, а использование этой переменной другой нитью, то возможно временное хранение нового значения в локальном представлении памяти первой нити и чтение его старого значения из общей памяти второй нитью.

Для принудительного согласования временной и общей памяти нити в OpenMP используется директива `flush`, которая позволяет определить точку синхронизации, в которой системой должно быть обеспечено единое для всех потоков состояние памяти (т. е. если потоком какое-либо значение извлекалось из памяти для модификации, то измененное значение обязательно должно быть записано в общую память). Следует отметить, что директива `flush` неявным образом присутствует в директивах `barrier`, `critical`, `ordered`, `parallel`, `for`, `sections`, `single`

```
#pragma omp flush [ (list) ]
```

Директива приостанавливает выполнение нити до тех пор, пока не завершатся все незавершенные к этому времени обмены с общей памятью, касающиеся перечисленных в списке переменных.

Неявно `flush` без параметров присутствует:

- в директиве `barrier`,
- на входе и выходе областей действия директив `parallel`, `critical`, `ordered`,
- на выходе областей распределения работ, если не используется опция `nowait`,
- в вызовах функций `omp_set_lock()`, `omp_unset_lock()`, `omp_test_lock()`, `omp_set_nest_lock()`, `omp_unset_nest_lock()`, `omp_test_nest_lock()`, если при этом замок устанавливается или снимается,
- перед порождением и после завершения любой задачи (`task`),
- вызывается для переменной, участвующей в операции, ассоциированной с директивой `atomic`.
- При выходе из секции директивы `for`, `single` или `sections`, если к директиве применено выражение `nowait` (вместе с неявным барьером эта директива убирает и неявную директиву `flush`).

Atomic

Данная директива относится к идущему непосредственно за ней оператору присваивания (на используемые конструкции накладываются некоторые ограничения), гарантируя корректную работу с общей переменной, стоящей в его левой части. На время выполнения оператора блокируется доступ к данной переменной всем запущенным в данный момент нитям, кроме нити, выполняющей операцию.

Атомарной является только работа с переменной в левой части оператора присваивания, при этом вычисления в правой части не обязаны быть атомарными.

```
#pragma omp atomic [read | write | update  
| capture]
```

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[])
{
    int count = 0;
    #pragma omp parallel
    {
        #pragma omp atomic
        count++;
    }
    printf("число нитей: %d", count);
}
```

В результате выполнения кода не происходит гонки, так как операция инкремент происходит атомарно.

Синхронизация с помощью замков

Один из вариантов синхронизации в OpenMP реализуется через механизм замков (locks). В качестве замков используются общие переменные. Данные переменные должны использоваться только как параметры примитивов синхронизации. Замок может находиться в одном из трёх состояний: неинициализированный, разблокированный или заблокированный. Разблокированный замок может быть захвачен некоторой нитью. При этом он переходит в заблокированное состояние. Нить, захватившая замок, и только она может его освободить, после чего замок возвращается в разблокированное состояние.

Есть два типа замков: простые замки и множественные замки. Множественный замок может многократно захватываться одной нитью перед его освобождением, в то время как простой замок может быть захвачен только однажды. Для множественного замка вводится понятие коэффициента захваченности (`nesting count`). Изначально он устанавливается в ноль, при каждом следующем захватывании увеличивается на единицу, а при каждом освобождении уменьшается на единицу. Множественный замок считается разблокированным, если его коэффициент захваченности равен нулю.

Для инициализации простого или множественного замка используются соответственно функции `omp_init_lock()` и `omp_init_nest_lock()`.

```
void omp_init_lock(omp_lock_t *lock);  
void omp_init_nest_lock(omp_nest_lock_t  
*lock);
```

После выполнения функции замок переводится в разблокированное состояние. Для множественного замка коэффициент захваченности устанавливается в ноль.

Для захватывания замка используются функции `omp_set_lock()` и `omp_set_nest_lock()`.

```
void omp_set_lock(omp_lock_t *lock);  
void omp_set_nest_lock (omp_nest_lock_t  
*lock);
```

Вызвавшая эту функцию нить дожидается освобождения замка, а затем захватывает его. Замок при этом переводится в заблокированное состояние. Если множественный замок уже захвачен данной нитью, то нить не блокируется, а коэффициент захваченности увеличивается на единицу.

Для освобождения замка используются функции `omp_unset_lock()` и `omp_unset_nest_lock()`.

```
void omp_unset_lock(omp_lock_t *lock);  
void omp_unset_nest_lock(omp_lock_t  
*lock);
```

Вызов этой функции освобождает простой замок, если он был захвачен вызвавшей нитью. Для множественного замка уменьшает на единицу коэффициент захваченности. Если коэффициент станет равен нулю, замок освобождается. Если после освобождения есть нити, заблокированные на операции, захватывающей данный замок, он будет сразу захвачен одной из ожидающих нитей.

```
omp_lock_t lock;
int n;
omp_init_lock(&lock);
#pragma omp parallel private (n)
{
    n = omp_get_thread_num();
    omp_set_lock(&lock);
    printf("Начало закрытой секции, %d\n", n);
    sleep(5);
    printf("Конец закрытой секции, %d\n", n);
    omp_unset_lock(&lock);
}
omp_destroy_lock(&lock);
```

Для неблокирующей попытки захвата замка используются функции `omp_test_lock()` и `omp_test_nest_lock()`.

```
int omp_test_lock(omp_lock_t *lock);  
int omp_test_nest_lock(omp_lock_t  
*lock);
```

Данная функция пробует захватить указанный замок. Если это удалось, то для простого замка функция возвращает 1, а для множественного замка – новый коэффициент захваченности. Если замок захватить не удалось, в обоих случаях возвращается 0.

```
omp_lock_t lock;
int n;
omp_init_lock(&lock);
#pragma omp parallel private (n)
{
    n = omp_get_thread_num();
    while (!omp_test_lock (&lock)) {
        printf("Секция закрыта, %d\n", n);
        sleep(2);
    }
    printf("Начало закрытой секции, %d\n", n);
    sleep(5);
    printf("Конец закрытой секции, %d\n", n);
    omp_unset_lock(&lock);
}
omp_destroy_lock(&lock);
```